

Reinforcement learning We also use the automatically constructed PRM to supervise LLMs with step-by-step PPO. In this scenario, we evaluate the accuracy of the LLMs’ greedy decoding output. An enhanced reward model is instrumental in training higher-performing LLMs.

3.2 REWARD MODELS FOR MATHEMATICAL PROBLEM

ORM Given a mathematical problem p and its solution s , ORM ($P \times S \rightarrow \mathbb{R}$) assigns a single real-value to s to indicate whether s is correct. ORM is usually trained with a cross-entropy loss (Cobbe et al., 2021; Li et al., 2023b):

$$\mathcal{L}_{ORM} = y_s \log r_s + (1 - y_s) \log(1 - r_s), \quad (1)$$

where y_s is the golden answer of the solution s , $y_s = 1$ if s is correct, otherwise $y_s = 0$. r_s is the sigmoid score of s assigned by ORM. The success of the reward model hinges on the effective construction of the high-quality training dataset. As the math problem usually has a certain answer, we can automatically construct the training set of ORM by two steps: 1) sampling some candidate solutions for a problem from a generator; 2) assigning the label to each sampling solution by checking whether its answer is correct. Although *false positives* solutions that reach the correct answer with incorrect reasoning will be misgraded, previous studies have proven that it is still effective for training a good ORM (Lightman et al., 2023; Yu et al., 2023a).

PRM Take a step further, PRM ($P \times S \rightarrow \mathbb{R}^+$) assigns a score to each reasoning step of s , which is usually trained with:

$$\mathcal{L}_{PRM} = \sum_{i=1}^K y_{s_i} \log r_{s_i} + (1 - y_{s_i}) \log(1 - r_{s_i}), \quad (2)$$

where y_{s_i} is the golden answer of s_i (the i -th step of s), r_{s_i} is the sigmoid score of s_i assigned by PRM and K is the number of reasoning steps for s . (Lightman et al., 2023) also conceptualizes the PRM training as a three-class classification problem, in which each step is classified as either ‘good’, ‘neutral’, or ‘bad’. In this paper, we found that there is not much difference between the binary and the three classifications, and we regard PRM training as the binary classification. Compared to ORM, PRM can provide more detailed and reliable feedback (Lightman et al., 2023). However, there are currently no automated methods available for constructing high-quality PRM training datasets. Previous works (Uesato et al., 2022; Lightman et al., 2023) typically resort to costly human annotations. While PRM manages to outperform ORM (Lightman et al., 2023), the annotation cost invariably impedes both the development and application of PRM.

3.3 AUTOMATIC PROCESS ANNOTATION

In this section, we propose an automatic process annotation framework to mitigate the annotation cost issues associated with PRM. We first define the quality of a reasoning step, followed by the introduction of our solution that obviates the necessity for human annotation.

3.3.1 DEFINITION

Inspired by Monto Carlo Tree Search (Kocsis & Szepesvári, 2006; Coulom, 2006; Silver et al., 2016; Świechowski et al., 2023), we define the quality of a reasoning step as *its potential to deduce the correct answer*. This criterion stems from the primary objective of the reasoning process, which essentially is a cognitive procedure aiding humans or intelligent agents in reaching a well-founded outcome (Huang & Chang, 2023). Therefore, a step that has the potential to deduce a well-founded result can be considered a good reasoning step. Analogous to ORM, this definition also introduces some degree of noise. Nevertheless, we find that it is beneficial for effectively training a good PRM.

3.3.2 SOLUTION

Completion To quantify and estimate the *potential* for a give reasoning step s_i , as shown in Figure 2, we use a ‘completer’ to finalize N subsequent reasoning processes from this step: $\{(s_{i+1,j}, \dots, s_{K_j,j}, a_j)\}_{j=1}^N$, where a_j and K_j are the decoded answer processes and the total number of steps for the j -th finalized solution, respectively. Then, we estimate the potential of this step based on the correctness of all decoded answers $A = \{a_j\}_{j=1}^N$.

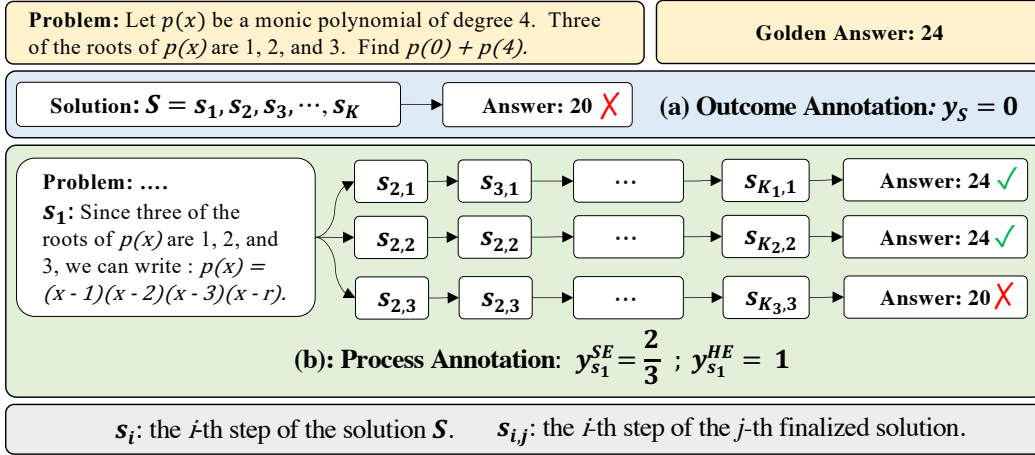


Figure 2: Comparison for previous automatic outcome annotation and our automatic process annotation. (a): automatic outcome annotation assigns a label to the entire solution S , dependent on the correctness of the answer; (b) automatic process annotation employs a ‘completer’ to finalize N reasoning processes ($N=3$ in this figure) for an intermediate step (s_1 in this figure), subsequently use hard estimation (HE) and soft estimation (SE) to annotate this step based on all decoded answers.

Estimation In this paper, we use two methods to estimate the quality y_{s_i} for the step s_i , hard estimation (HE) and soft estimation (SE). HE supposes that a reasoning step is good as long as it can reach the correct answer a^* :

$$y_{s_i}^{HE} = \begin{cases} 1 & \exists a_j \in A, a_j = a^* \\ 0 & \text{Otherwise} \end{cases} \quad (3)$$

SE assumes the quality of a step as the frequency with which it reaches the correct answer:

$$y_{s_i}^{SE} = \frac{\sum_{j=1}^N \mathbb{I}(a_j = a^*)}{N}. \quad (4)$$

Once we gather the label of each step, we can train PRM with the cross-entropy loss. In conclusion, our automatic process annotation framework defines the quality of a step as its potential to deduce the correct answer and achieve the label of each step by completion and estimation.

3.4 RANKING FOR VERIFICATION

Following (Lightman et al., 2023), we use the minimum score across all steps to represent the final score of a solution assigned by PRM. We also explore the combination of self-consistency and reward models following (Li et al., 2023b). In this context, we initially classify solutions into distinct groups according to their final answers. Following that, we compute the aggregate score for each group. Formally, the final prediction answer based on N candidate solutions is:

$$a_{sc+rm} = \arg \max_a \sum_{i=1}^N \mathbb{I}(a_i = a) \cdot RM(p, S_i). \quad (5)$$

Where $RM(p, S_i)$ is the score of the i -th solution assigned by ORM or PRM for problem p .

3.5 REINFORCE LEARNING WITH PROCESS SUPERVISION

Upon achieving PRM, we employ reinforcement learning to train LLMs. We implement Proximal Policy Optimization (PPO) in a step-by-step manner. This method differs from the conventional strategy that utilizes PPO with ORM, which only offers a reward at the end of the response. Conversely, our step-by-step PPO offers rewards at the end of each reasoning step.

Models	Verifiers	GSM8K	MATH500
LLaMA2-70B: MetaMATH	Self-Consistency	88.0	39.4
	ORM	91.8	40.4
	Self-Consistency + ORM	92.0	42.0
	MATH-SHEPHERD (Ours)	93.2	44.5
	Self-Consistency + MATH-SHEPHERD (Ours)	92.4	45.2
Llemma-34B: MetaMATH	Self-Consistency	82.6	44.2
	ORM	90.0	43.7
	Self-Consistency + ORM	89.6	45.4
	MATH-SHEPHERD (Ours)	90.9	46.0
	Self-Consistency + MATH-SHEPHERD (Ours)	89.7	47.3
DeepSeek-67B: MetaMATH	Self-Consistency	88.2	45.4
	ORM	92.6	45.3
	Self-Consistency + ORM	92.4	47.0
	MATH-SHEPHERD (Ours)	93.3	47.0
	Self-Consistency + MATH-SHEPHERD (Ours)	92.5	48.1

Table 1: Performances of different LLMs on GSM8K and MATH with different verification strategies. The reward models are trained based on LLaMA2-70B and Llemma-34B on GSM8K and MATH, respectively. The verification is based on 256 outputs.

4 EXPERIMENTS

Datasets We conduct our experiments using two widely used math reasoning datasets, GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). For the GSM8K dataset, we leverage the whole test set in both verification and reinforcement learning scenarios. For the MATH dataset, in the verification scenario, due to the computation cost, we employ a subset MATH500 that is identical to the test set of Lightman et al. (2023). The subset consists of 500 representative problems, and we find that the subset evaluation produces similar results to the full-set evaluation. To assess different verification methods, we generate 256 candidate solutions for each test problem. We report the mean accuracy of 3 groups of sampling results. In the reinforcement learning scenario, we use the whole test set to evaluate the model performance. We train LLMs with MetaMATH (Yu et al., 2023b).

Parameter Setting Our experiments are based on a series of large language models, LLaMA2-7B/13B/70B (Touvron et al., 2023), Llemma-7B/34B (Azerbayev et al., 2023), Mistral-7B (Jiang et al., 2023) and DeepSeek-67B (DeepSeek, 2023). We train the generator and completer for 3 epochs on MetaMATH. We train the Mistral-7B with a learning rate of $5e-6$. For other models, The learning rates are set to $2e-5$, $1e-5$, and $6e-6$ for the 7B/13B, 34B, and 67B/70B LLMs, respectively. To construct the training dataset of ORM and PRM, we train 7B and 13B models for a single epoch on the GSM8K and MATH training sets. Subsequently, we sample 15 solutions per problem from each model for the training set. Following this, we eliminate duplicate solutions and annotate the solutions at each step. We use Llemma-7B as the completer with the decoded number $N=8$. Consequently, we obtain around 170k solutions for GSM8K and 270k solutions for MATH. For verification, we choose LLaMA2-70B and Llemma-34B as the base models to train reward models for GSM8K and MATH, respectively. For reinforcement learning, we choose Mistral-7B as the base model to train reward models and use it to supervise LLaMA2-7B and Mistral-7B generators. The reward model is trained in 1 epoch with a learning rate $1e-6$. For the sake of convenience, we train the PRM using the hard estimation version because it allows us to utilize a standard language modeling pipeline by selecting two special tokens to represent ‘has potential’ and ‘no potential’ labels, thereby eliminating the need for any specific model adjustments. In reinforcement learning, the learning rate is $4e-7$ and $1e-7$ for LLaMA2-7B and Mistral-7B, respectively. The Kullback-Leibler coefficient is set to 0.04. We implement a cosine learning rate scheduler, employing a minimal learning rate set to $1e-8$. We use 3D parallelism provided by hfai¹ to train all models with the max sequence length of 512.

¹<https://doc.hfai.high-flyer.cn/index.html>